

Simulation Framework

Activity Options

Type	Description
Matrix or Cascading Dropdowns	- Optimize settings using a slider on a number line or a set of sliders along Cartesian axes. - Dropdowns gray out or change based on upstream selections mirror that model. Comparative Analysis can point out hierarchy errors by showing correct and incorrect structure side by side.
Decision Trees	In a realistic scenario, the learner makes a sequence of decisions – where each choice produces a consequence that shapes the next decision point to show the learner what happens downstream. This is useful for troubleshooting workflows and incident response.
Before / After Comparison	Present a degraded or misconfigured system, which needs to make targeted changes to impact output or behavior. The real value is highlighting the delta – how much it moved the needle and whether it introduced any new tradeoffs. This is especially effective for optimization and performance tuning scenarios, where the relationship between a configuration change and its outcome is non-obvious.
Logic Puzzle	Present a target outcome and a set of rules in a logic puzzle format. The learner must configure a system to meet the goal without violating any constraints. This works well for capacity planning, access control design, and network or data architecture problems. It's more goal-directed than open exploration and better mirrors the actual experience of working within real system limitations.
Role Play	Simulates a customer or internal conversation. The learner is a support engineer, solutions architect, or implementer who must ask diagnostic questions, interpret the answers, and propose a solution to develop the communication and discovery skills that accompany technical judgment. This works well for pre-sales, customer-facing technical roles, and requirements gathering.
System Interpretation	Provide a realistic system output – log file, API response, monitoring dashboard snapshot, or stack trace which the user leverages to determine: what happened, in what order, and what should happen next. This mirrors a common real-world task.
Comparative Analysis	Show two systems or configurations side-by-side. The learner must identify what differs, predict how each will behave under a given condition, and explain which is preferable and why. This develops the evaluative judgment needed for reviewing architecture proposals, choosing between implementation approaches, or assessing whether a setup follows best practices.
Configuration Sandbox	A constrained environment where the learner configures, tests, and troubleshoots a system – set parameters, define rules, connect services. This is great for learners to apply foundational knowledge, but requires the most effort to create and maintain.

In general, sliders and dropdowns are well suited to *discovery and exploration*. The others focus on *applied judgment, diagnostic reasoning, and evaluation*.

Simulation Patterns

Type	Great Options	Example
Hierarchical Relationship	Cascading Dropdowns or Logic Puzzles Demonstrate fundamental rules that propagate across levels.	Role-Based Access Conflict Resolver <i>example:</i> Enterprise software users can have multiple roles—a Viewer role at the organization level and an Editor role on a specific project. Which wins? <i>Activity:</i> The learner assigns roles via two dropdowns (org, project level). A third option enables the user to set the desired action (read, edit, delete, share). The output shows whether access is granted and which rule resolved the conflict. The learner cycles through combinations to discover the precedence logic. <i>Assessment:</i> A short scenario describes an unintended access outcome. The learner must select which combination of role assignments caused it and propose the corrected configuration. <i>Impact:</i> Role conflicts are one of the most common real-world misconfiguration problems and hard to see. Rather than memorizing a precedence table, the learner discovers the logic through exploration, which leads to durable understanding. <i>Similar Examples:</i> Cascading file system permissions, programming object inheritance.
Dependent Relationship	Cascading Dropdowns, System, Interpretation, or Configuration Sandbox Determine what enables the flow to continue or branch.	API Parameter Dependency Tester <i>example:</i> A REST API call where a parameter is only required when specific optional parameters are included — this conditional dependency isn't obvious from the documentation. <i>Activity:</i> Construct valid requests for different use cases. 1) Use checkboxes to turn on specific parameters and enter data for them. 2) Select the "Send Request" button 3) Based on the inputs, the result displays: success, partial success, or a specific error code with a simple explanation of the dependency rule. <i>Assessment:</i> Present a pre-filled API request with one subtle misconfiguration. The learner must identify the invalid combination, explain why it's invalid, and correct it. <i>Impact:</i> API dependencies are difficult to discern from documentation. Experiencing the error responses in context, and correcting them, builds the pattern recognition needed for success. <i>Similar Examples:</i> workflows with specific steps, decision trees
Interoperability	2x2 Matrix or Configuration Sandbox Focus on what happens at the edge when you vary settings. Compare a working	Configuration Compatibility Matrix <i>example:</i> Connect a 3rd-party identity provider (IdP) to a SaaS app, where the authentication protocol, token format, and session timeout settings need to align between systems. <i>Activity:</i> A 2x2 matrix with sliders on each axis—one represents settings on System A (e.g., token expiry time 5–60 min), the other is the corresponding setting on System B (e.g., session timeout 5–60 min). As the learner moves the sliders, the output displays green (compatible), yellow (degraded), and red (failure). Additional dropdowns let the learner change additional settings to adjust slider ranges to green. <i>Assessment:</i> From a sample configuration, predict if a configuration will succeed, grade, or fail. <i>Impact:</i> Interoperability failures often degrade gracefully before failing. The slider makes that gradient visible and explorable.

	integration configuration with a broken one.	Simulation B: Dataflow Routing Simulator <i>example:</i> A middleware layer receives data from a source system, transforms it, and passes it to a destination — where the transformation rules must account for differences in field names and data types. <i>Activity:</i> Two panels show the data schema for Source and Destination. 1) The learner drags the fields between the two to show the connection and then selects a transformation rule for each mapping from a dropdown. 2) When the user selects Test Flow, it shows what transpires when a sample record goes through the system, including any mapping errors. <i>Assessment:</i> Show a partial mapping with errors. Ask the learner to identify and fix. <i>Impact:</i> Data mapping is complex and abstract. This simulation helps make tangible connections to the theory.
Design Constraints	2x2 Matrix & Logic Puzzles Constraints are about limits and the interactions between them. Sliders help learners find the edges of those limits. Logic puzzles multiple constraints simultaneously.	Performance Boundary Explorer <i>example:</i> A message queue system with a max throughput limit. When producers send messages faster than consumers can process, the queue backs up and eventually drops messages. <i>Activity:</i> A 2x2 matrix with sliders. The X-axis controls message production rate (msg/sec); Y-axis controls # of threads. As the learner adjusts the sliders, the display updates the queue depth, processing latency, and message drop rate. Color-coded zones indicate optimal, strained, and failure states. <i>Assessment:</i> A scenario where there is intermittent message loss during peak hours. Give the current slider settings and ask “Which variable is the cause?” “What adjustment would resolve it?”. <i>Impact:</i> Documentation generally lists constraints (“max throughput: 1,000 msg/sec”), but not necessarily potential variable interactions. Understanding the interaction between key variables builds a mental model for real-world tuning decisions. Alternative activity: Scaling Limits Decision Tree <i>example:</i> A database with hard limits on connections, storage, and query complexity — where exceeding any one of them triggers different failure modes. <i>Activity:</i> A set of dropdowns representing a deployment configuration — # of concurrent users, avg. query complexity, data volume, and replication factor. Each selection narrows the downstream dropdowns (some combinations become grayed out). The final output summarizes the limits of the configuration approaches, which it exceeds, and what failure mode each would trigger. The learner must find a configuration that satisfies a given set of requirements without hitting any limits.
Patterns and Anti-patterns	2x2 Matrix, System Interpretation, and Comparative Analysis	System Health Anomaly Detector <i>example:</i> An app monitoring dashboard contains: CPU usage, memory consumed, error rate, and response time. Only certain combinations indicate real problems. <i>Activity:</i> A 2x2 matrix with sliders controls four metrics simultaneously, two per axis. The learner adjusts the sliders to match normal scenarios (“Morning traffic is ramping up normally”) and those to diagnose an active incident, and the most likely cause. <i>Assessment:</i> Show the learner metric snapshots side by side. Ask them to rank the snapshots by severity, identify which represents an anti-pattern specifically, and explain what distinguishes it from the others. Alternatively, display a set of symptom cards and a set of root cause cards and ask the learner to match them. <i>Impact:</i> Pattern recognition requires seeing enough examples of the full signal, including normal variations that could be mistaken for a problem, to accurately interpret information and develop judgement.

Simulation Format Decision Guide

Generally you'll select a primary format as the anchor activity and use others (primary or secondary) as either a warm-up to build context prior to it or as a follow-up where the learner applies what they discovered.

	Hierarchical Relationships	Dependent Relationships	Interoperability	Design Constraints	Patterns and Anti-Patterns
2×2 Matrix with Sliders	Secondary	Secondary	Primary	Primary	Secondary
Cascading Dropdowns	Primary	Secondary	Secondary	Primary	—
Decision Tree	Secondary	Primary	Secondary	Secondary	Primary
Constraint Puzzle	Primary	Secondary	—	Primary	Secondary
Before / After Comparison	Secondary	Secondary	Primary	Primary	Secondary
Role-Play	Secondary	Primary	Secondary	—	Secondary
Log & Output Interpretation	—	Primary	Secondary	Secondary	Primary
Build & Test Sandbox	Secondary	Primary	Primary	Secondary	Secondary
Comparative Analysis	Primary	—	Primary	Secondary	Primary

Activity sequencing recommendation

1. Use a format that lets the learner explore freely with low stakes — typically the 2×2 Matrix, Cascading Dropdowns, or Build-and-Test Sandbox. The goal is to build an accurate mental model through discovery.
2. Raise the stakes using a realistic situation where the users have to apply the mental model, such as Scenario Branching or Role-Play.
3. Ask the learner to evaluate and explain, using Comparative Analysis or a Constraint-Based Puzzle with a reflection prompt. This is where understanding consolidates into judgment, which is the level of capability that actually transfers to the job.
4. CTA: Targeted action (via an assistant, within community, or in-app).

Vibe Code Prototyping Tips

Design

- Focus on your educational goals, rather than styling or details.
 - Determine if you will have one simulation, or a series of activities.
 - Code and test them separately.
- Put your styling / design standards into a separate resource file that can be leveraged by your AI tool. This improves efficiency and consistency, but also gives you more control over the context window for your simulation.
 - This can include the exact hex codes and UI patterns that match your corporate identity. They may already exist as a resource you can leverage.

Coding and Testing

- Break up your projects into small tasks to keep the AI from losing the thread of the UI logic.
- Be as clear and precise in your directions as possible, including giving examples of simulations that you like (or even things to avoid).
 - Upload existing simulations that are solid and what it is that you think make them work well.
 - If you see an example you like, upload it and ask the AI to describe it back to you.
 - If the AI breaks a layout or misses a CSS class, you will have to tell it to fix itself. It might inspire ideas you haven't had.
- AI tools vary. They change often and they're becoming more advanced. Try multiple tools, you'll get different results. Retest over time.
- Build reusable prompts that you refine. And, create custom prompts for tailored results.
- Be comfortable with the output structure. Wherever you're going to "land", whatever the output will be; if you'll get HTML, know HTML. You don't have to be an expert, but it is very helpful to troubleshoot and clean up.

Output

Understand your output options relative to where you want to use the simulation:

- Is it part of a traditional SCORM course?
 - Yes. What is the context? Will the simulation include the context or will it be in the course?
- Do you want to embed it in other content (in-app, doc, kb articles, etc.)?
 - Are there constraints in the target system(s)?
 - Where will you host the simulation?
 - What format will it be in?
 - Will you wrap it with other content (e.g., Parta, HTML, etc.)